Future risk prediction - model card and dataset provenance

Artefact: `app/src/main/assets/future_risk_model.json` (also `ml/outputs/future_risk/future_risk_model.json`) **Model version string:** `future-risk-unimib-v1` **Pipeline:** `ml/future_risk/` (`config.py`, `features.py`, `dataset.py`, `train.py`, `tune.py`) **This card describes:** a *trained, supervised, future-event* model. It is not the geofence rule, and it is not the approach-risk checklist.

Every number below was written by `ml/future_risk/train.py` into `ml/outputs/future_risk/metrics.json`, `dataset_provenance.json` and `tuning.json`, and is re-checked by `ml/future_risk/tests/test_future_risk.py`. Nothing here is estimated, rounded up, or carried over from an earlier run.

---

1. What the model answers

> Given the **last second** of phone accelerometer data, will a **hard fall impact** happen within the **next second**?

Both halves of that sentence are load-bearing:

- **"hard fall impact"** is a real, physical, recorded event in a public dataset - not a proxy, not a score, not a label invented by this project.
- **"within the next second"** is a horizon the data can actually support (section 4). It is **not** a 30-minute forecast, and a 30-minute forecast is not attempted, because accelerometer data cannot see 30 minutes into the future.

In scope

- An informational signal to the tourist ("the model thinks something hard may land in the next second") and a throttled summary to the authority dashboard.
- A pre-impact **fall** forecast, for phones carried on the body, held in the hand, or in a pocket - the placements UniMiB SHAR actually recorded.

Out of scope - explicitly

Not claimed	Why
Any horizon longer than 1 s	Not learnable from accelerometer data (section 4).
Tourist incidents, injuries, crime, medical events, or "danger zones"	The training labels are **falls in a laboratory**, not tourist incidents.
A standalone fall alarm	Measured false-alarm rates are far too high to alarm on (sections 9 and 10).
Causation of any kind	The reported contributing factors are "readings this model leans on", nothing more.
Any use as a safety-critical system	This has never been validated on a real deployment, on real tourists, or on a phone in a moving vehicle.
Automatic SOS	Risk **never** triggers an SOS. SOS remains a deliberate tourist action. See section 12.

---

2. Dataset and provenance

UniMiB SHAR - Micucci, D., Mobilio, M., Napoletano, P. (2017). *UniMiB SHAR: A Dataset for Human Activity Recognition Using Acceleration Data from Smartphones.* arXiv:1611.07688v2. University of Milano-Bicocca.

Fact	Measured value
Archive	`ml/data/unimib/UniMiB-SHAR.zip`
Archive size	266,206,080 bytes
Archive SHA-256	`365a396729c0755b4ba8ef8ee0ba52a0621aa535282470546d8a663ff2567160`
Source file used	`data/full_data.mat` (29,084,022 bytes), SHA-256 recorded in `dataset_provenance.json`
Subjects	30 (gender, age, height, weight recorded for all 30)
Activities	9 of daily living + 8 fall types
Sampling rate	50 Hz
Trials read	1,980
Trials used	**1,979** (1 dropped - see below)
Units	g (total acceleration, gravity included)

Why `full_data.mat` and not the shipped windows

UniMiB also ships pre-cut 151-sample windows. They are **centred on the peak of the impact**, so using them would destroy the pre-impact timeline this model's label is built from - there would be no "before the impact" left to predict from. The pipeline therefore reads `full_data.mat`, the full recordings, and cuts its own windows.

Licence position, stated as it is

UniMiB SHAR is publicly distributed by its authors for academic research use. **The download carries no explicit OSI-approved licence.** No licence is asserted in the provenance file; downstream reuse should cite the paper and treat the terms as research-use-only. Only derived artefacts are redistributed here - model weights, aggregate metrics and 16 reference windows - never the raw recordings.

The defect that was found, and what was done about it

One recording is corrupt: **subject 10, `Jumping`, trial 1** contains **9,369 non-finite values out of 9,624** (3,208 samples). It was **dropped, counted and reported** (`data_quality.dropped` in `dataset_provenance.json`) - not interpolated, not filled with the mean, and not silently skipped. A NaN in a feature vector would have propagated into every tree that touched it. This is why the kept-trial count is 1,979 rather than 1,980, and why `Jumping` shows 59 trials in the per-activity table instead of 60.

Data-quality note on the subject table

Two subjects' height/weight fields in the MATLAB struct are unpopulated; the pipeline treats "unreadable" and "zero" differently and never substitutes a population mean for a missing body measurement. `dataset_provenance.json` records the subject table verbatim for audit.

---

3. Target and label definition

Item	Value
Positive class	`IMMINENT_FALL`
Negative class	`NO_IMMINENT_FALL`
Horizon	**50 samples = 1.0 s**
Feature window	**50 samples = 1.0 s**
Window stride	10 samples (0.2 s)

Item	Value
Impact threshold	peak magnitude >= **2.0 g**

How the impact instant is found

For each fall trial, `t_impact = argmax(magnitude over the trial)`, accepted only if `magnitude[t_impact] >= 2.0 g`.

- The `argmax` is a **post-hoc** quantity: the impact *time* is only knowable after the recording. That is what supervised labelling means, and the provenance file says so in as many words.
- 2.0 g reuses the app's own `FallPolicy.MIN_IMPACT_G`, so the model and the heuristic agree on what a hard landing is.
- **6 fall trials** had no sample reaching 2.0 g. They are excluded from the positives, counted in `window_stats.n_fall_trials_without_impact`, and reported.
- Over the 1,440 fall trials used, the impact peak ranged from **2.0299 g** to **3.4562 g**.

The labelling rule

- **Positive:** a window ending at sample `t` in `[t_impact - 50, t_impact - 1]`, i.e. a lead time of 1..50 samples.
- **Negative:** a window cut at the same 0.2 s cadence from an **activity-of-daily-living** trial.

The negative definition is deliberately the narrow one. A post-impact window is *not* a training negative - "the fall has already landed" is a different question, and mixing it in would blur what the negative class means. Post-impact windows are instead kept aside as a **diagnostic** (section 9).

Why the two classes have an honest meaning

A positive really is "a recorded human fall is one second away". A negative really is "a recorded human is walking, running, sitting, standing, lying down, or climbing stairs". Neither class depends on anything from after its own prediction instant except the impact *time* used to build the label.

Leakage, checked rather than asserted

`train.check_no_future_information()` runs on every training pass and its result is stored in `metrics.json`:

Check	Result
Window length	50 samples
Horizon	50 samples

Check	Result
All positive windows end **before** their own impact	`true`
All leads within the horizon (1..50)	`true`
Feature window never overlaps the label window	`true`
Minimum / maximum lead observed	10 / 50 samples
Max end index over all positives	266

A window ending at `t` uses samples `[t-49, t]`; the label asks about `(t, t+50]`. The two intervals are disjoint by construction, and the test re-derives that from the raw recordings rather than trusting the summary.

The second leakage control is the **shuffled-label run**: the same features, the same pipeline and the same forest, with the labels randomly permuted. It scores **ROC-AUC 0.47743 / PR-AUC 0.129885** - chance. If the construction leaked, this number would exceed 0.5. It does not.

---

4. Why the horizon is one second

The horizon was **measured, not chosen for convenience**:

- Fall trials run 151-446 samples (median **174 samples = 3.5 s**).
- The impact sits around index ~120 in a median trial, leaving **>= 95 samples** of pre-impact signal.
- A 50-sample (1.0 s) window that must end at least 1 sample before the impact therefore fits in every included trial. Longer windows would not, which is why 1.0 s is the *longest fully supported* horizon and not merely a round number.

Lead-time distribution over the positives (windows per lead, 10-sample stride):

Lead (samples)	50	40	30	20	10
Windows	923	1,268	1,412	1,425	1,430

Explicitly not attempted: a 30-minute danger forecast. The four datasets named in `DATASETS.pdf` (GeoLife, UCI HAR, OpenStreetMap, OpenWeatherMap/Open-Meteo) contain **no incident or injury outcome at all**, so a supervised tourist-incident model cannot be trained from them. The honest options were to pick a dataset with real outcome labels, or to fabricate one. This pipeline picked the former and says so.

---

5. Features (22, in fixed order)

Extracted from **the accelerometer only**, in **g**, from the phone's own sensor. The order is the forest contract: every tree node stores a feature *index*, so reordering this list silently changes every prediction. `FutureRiskFeatures.NAMES` in Kotlin asserts the same list, and `FutureRiskReferenceTest` fails if they diverge.

#	Feature	Meaning
0	`acc_mag_mean`	Average force on the phone
1	`acc_mag_std`	How much the force is fluctuating
2	`acc_mag_min`	Lightest moment
3	`acc_mag_max`	Hardest moment
4	`acc_mag_range`	Spread between lightest and hardest
5	`acc_mag_energy`	Mean squared magnitude
6	`acc_mag_peak_jerk`	Largest single-sample change in force
7	`acc_mag_zcr`	How often the force crosses its own mean
8-10	`acc_x_mean`, `acc_y_mean`, `acc_z_mean`	Mean direction of the force
11-13	`acc_x_std`, `acc_y_std`, `acc_z_std`	Per-axis motion
14-16	`acc_xy_corr`, `acc_xz_corr`, `acc_yz_corr`	How the axes move together
17	`freefall_fraction`	Share of the second below 0.5 g
18	`tilt_deg`	Angle of the phone from vertical
19	`orientation_change_deg`	Rotation between the two halves of the second
20	`acc_mag_slope`	Direction of the trend in force
21	`acc_mag_late_to_early_ratio`	Late force over early force

Implementation details that matter for parity:

- Population standard deviation (`ddof = 0`).
- `np.signbit` semantics for the zero-crossing count (negative zero counts as negative).
- `freefall_fraction` uses the fixed threshold 0.5 g.
- Orientation features return **0** when the mean acceleration vector's norm is below 0.1 g, because the direction is undefined there - a division by ~zero would otherwise produce an arbitrary angle.

- The late/early ratio floors its denominator at 1e-3.

Missing and bad inputs

The model is only ever given a full 50-sample window. If fewer samples exist, the app reports **"Insufficient data"** and no band; it does not pad, interpolate or carry the last window forward as if it were fresh. If any sample is non-finite, or any magnitude exceeds **16 g** (which is what a driver reporting m/s^2 instead of g looks like), the app reports the reading as unusable and predicts nothing. The contract lives in the artefact as `insufficient_data` so the app and the pipeline cannot disagree about it.

Contributing factors, and the caveat that travels with them

The artefact ships the forest's global Gini importance, the training mean and standard deviation per feature, and a human-readable label per feature. The app ranks features by `importance x |z-score|`, keeps those with `|z| >= 0.75`, and shows at most 3.

> **These are the readings this model leans on that are currently unusual. They are not causes, and the model makes no claim that they are.**

That sentence is part of the app's card text and is asserted by a test, because a factor list is exactly the sort of thing a reader turns into a causal story.

---

6. Training methodology

Item	Value
Split	**subject-disjoint**, 60/20/20
Seed	42
Subjects	30 -> train 18 / validation 6 / test 6
Subjects disjoint	**true** (checked in `metrics.json`)
Windows	51,128 total: train 31,122 / val 9,591 / test 10,415
Positives	6,458 (12.63% of all windows)
Negatives	44,670

Item	Value
Imbalance handling	`class_weight="balanced_subsample"`
Preprocessing	none that needs fitting: every feature is scale-free per window (means, stds, correlations, angles, fractions, a ratio) - there is no scaler fitted on the training split, therefore nothing that could leak a global mean into the test set
Test set	touched **once**, after every choice was frozen

Subject-disjoint splitting is the only defensible choice for body-worn sensor data: the same person in both training and test would let the model memorise an individual's gait. Validation subjects `[12, 13, 15, 18, 24, 29]` were used for calibration, band thresholds and hyper-parameter selection; test subjects `[2, 3, 5, 9, 14, 28]` were not inspected until the final scoring pass.

Validation's only jobs: fitting the Platt calibration, choosing the two band thresholds, and choosing the hyper-parameters. Nothing was tuned on the test set.

---

7. Model selection under a size budget

`ml/future_risk/tune.py` searched **65 configurations** on the validation split only.

The winner by validation PR-AUC was rejected, and this is the honest reason:

Configuration	Val PR-AUC	Val ROC-AUC	Exported asset	Nodes
300 trees, unlimited depth, leaf 1, `sqrt`, balanced	**0.8759**	0.9764	**20,059.7 KB**	700,492
300 trees, depth 8, leaf 8, `max_features=0.3`	0.8201	0.9625	2,003,677 **bytes** (over budget by 3,677)	66,962
120 trees, depth 11, leaf 4, `sqrt`, balanced (shipped)	**0.8451**	**0.9709**	**1,968,314 bytes**	**66,894**

The **2,000,000-byte budget is a deployment constraint**: the forest is exported to JSON and walked on a phone from an app asset. It is not a quality metric, and the selection rule is recorded in `tuning.json` (`selection_rule`) and re-derived by a test rather than being a note in a document.

Two things worth reading off that table:

- Only **1 of 65** configurations fits the budget at all. The constraint is genuinely binding, not decorative.
- The shipped configuration is **both better and smaller** than the smallest 300-tree export (0.8451 vs 0.8201, 1,968,314 vs 2,003,677 bytes), so the tree budget is better spent on depth than on tree count here.

Cost of the constraint, stated plainly: **-0.031 validation PR-AUC** (0.8759 -> 0.8451), and it is the reason the shipped model is not the best model this pipeline can fit.

Shipped hyper-parameters live in `ml/future_risk/config.py` (`N_ESTIMATORS`, `MAX_DEPTH`, `MIN_SAMPLES_LEAF`, `MAX_FEATURES`, `CLASS_WEIGHT`) and are asserted against `tuning.json.selected` by the test suite, so a config edit that is not reflected in the search record fails a test.

---

8. Calibration

A forest's vote fraction is not a probability. The shipped model applies **Platt scaling** (logistic regression on `logit(p_forest)`), fitted **on the validation split only**:

```
calibrated = 1 / (1 + exp(-(a * ln(p/(1-p)) + b)))
a = 1.24940294   b = -1.07904853
logit clipped at 1e-6 (matches LOGIT_CLIP in FutureRiskModel.kt)
```

Metric (held-out test)	Raw forest	Calibrated
Brier score	0.062600	**0.056675**

Reliability on the held-out test set (10 bins, `calibration_reliability.csv`):

Bin	n	Mean predicted	Observed frequency
0.0-0.1	8,376	0.0122	0.0224
0.1-0.2	326	0.1450	0.1902
0.2-0.3	193	0.2482	0.3886
0.3-0.4	140	0.3461	0.5000
0.4-0.5	123	0.4482	0.4553
0.5-0.6	135	0.5517	0.4296
0.6-0.7	138	0.6520	0.5217

Bin	n	Mean predicted	Observed frequency
0.7-0.8	189	0.7504	0.5608
0.8-0.9	299	0.8589	0.6957
0.9-1.0	496	0.9407	0.8952

Read this table honestly. Calibration improves the Brier score and the top bin is close, but the middle bins (0.2-0.4) are badly over-confident: 140 windows predicted at ~0.35 were positive 50% of the time. The probability is therefore shown in the app **only because it is calibrated**, and it is shown alongside the band, the thresholds and the model version - never as a bare percentage.

---

9. Measured performance (held-out test set)

Test set: 10,415 windows from 6 subjects never seen in training, 1,339 positive.

Baselines and the shipped model

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Majority class	0.0000	0.0000	0.0000	0.5000	0.128565
Logistic regression	0.4760	0.8432	0.6085	0.904389	0.659633
Hist gradient boosting	**0.7916**	0.6721	**0.7270**	**0.945913**	**0.815000**
Random forest, raw vote @ 0.5	0.6534	0.7827	0.7122	0.936848	0.764569
Random forest, calibrated, HIGH band >= 0.314046 (shipped)	**0.6707**	**0.7498**	**0.7080**	**0.936848**	**0.764569**

The majority-class row is not padding: with a 12.86% positive rate, a model that always says "no" scores 87.1% accuracy, and PR-AUC 0.128565 *is* that positive rate. Accuracy alone would be a lie here, which is why every table in this card reports precision, recall, PR-AUC and ROC-AUC.

The shipped model is not the best model available - and that is recorded on purpose

`hist_gradient_boosting` scores **PR-AUC 0.8150** against the shipped forest's **0.7646**, and it also wins on precision, F1 and ROC-AUC. It is **not shipped**, because it cannot be walked on-device by `DecisionForest` and its serialisation does not fit the asset budget. Anyone comparing this model against the number 0.8150 should know that a better offline model exists in the same repository, and can be produced with one command.

Confusion matrix, shipped model at the HIGH threshold

Labels: `NO_IMMINENT_FALL`, `IMMINENT_FALL`.

	Predicted no	Predicted yes
Actually no	8,583	493
Actually yes	335	1,004

Accuracy 0.9205, Brier 0.056675, threshold 0.314046.

Detection over the horizon (the lead-time table)

Lead	Positive windows	Detected	Detection rate
0.2 s	284	204	0.718310
0.4 s	281	199	0.708185
0.6 s	281	217	0.772242
0.8 s	270	213	0.788889
1.0 s	223	171	0.766816

Detection does **not** collapse as the lead grows, which is the interesting result: there is real pre-impact signal, not just "the last moment before the peak".

Trial-level sensitivity - the number that matters most

Metric	Value
Held-out fall trials	**284**
Trials with at least one window flagged HIGH	**264**
Trial sensitivity	**0.929577**

This is reported separately from window recall (0.7498) because a single fall generates many overlapping windows, so window recall flatters any model that catches one of them. **"93% of real falls were flagged at least once within the last second"** is the honest headline; 0.7498 is the window-level number underneath it.

False alarms on ordinary activity

Metric	Value
ADL windows on the test subjects	9,076
ADL windows flagged HIGH	493
ADL window false-positive rate	0.054319
False alarms per minute of ordinary activity (0.2 s stride)	**16.295725**
False alarms per minute at the app's 0.64 s evaluation cadence	**5.092414**
ADL seconds evaluated	1,815.2

A minute of walking, sitting, standing, lying down or stair-climbing produces **about 5 HIGH-band windows** at the app's cadence. **This is the single most important limitation in this card.** A HIGH band is an *informational UI state* and a queued summary; it is **not** an alarm, and this model is **not a deployment-ready** standalone fall alarm.

Per-activity false-alarm rates, so the limitation is visible rather than averaged away:

Activity	Rate	Windows	Alarms
Walking	0.011076	1,896	21
Running	0.016412	1,767	29
Going upstairs	0.025335	671	17
Going downstairs	0.044248	678	30
Jumping	**0.160615**	716	115
Sitting down	**0.156385**	697	109
Standing up (from sofa)	0.027586	725	20
Lying down (from sofa)	**0.127368**	950	121
Standing up (from floor)	0.031762	976	31

The mechanism is visible in the table: the model's three worst confusions (jumping, sitting down, lying down) are the three activities where a body genuinely *drops* under gravity. Walking and running - the activities with the most windows - are the safest. A user who jumps, sits or lies down a lot will see noticeably more moderate/high bands.

Post-impact diagnostic

Windows taken *after* a real impact landed are not training data, but they are scored as a diagnostic: 970 windows, 72 alarms, rate **0.074227**. A landed fall sits still and then gets up, so this is well behaved - but this number is a *diagnostic*, not a test-set metric, and it is labelled as such in `metrics.json`.

Figures (`ml/outputs/future_risk/figures/`)

`fig_01_precision_recall.png`, `fig_02_roc.png`, `fig_03_calibration.png`, `fig_04_confusion_matrix.png`, `fig_05_lead_time.png`, `fig_06_example_windows.png`.

---

10. Risk bands, and why they are defensible

Thresholds are **fitted on the validation split** (9,591 windows, positive rate 0.129392) and then frozen:

Band	Rule	Threshold
HIGH	the threshold maximising F1 on validation	**>= 0.314046**
MODERATE	the **largest** threshold whose validation recall is still >= 0.95, capped at HIGH	**>= 0.069201**
LOW	below MODERATE	-

Both numbers travel inside the model artefact and are shown in the app with the sentence that produced them ("fitted on held-out validation subjects"). The rule is in `metrics.json` (`risk_bands.rule`) and in the artefact, and the "largest threshold" search is asserted by a test - the earlier draft of this code took the *first* qualifying threshold, which silently collapsed MODERATE onto HIGH and made the middle band unreachable. That bug is now covered.

`UNAVAILABLE` is not a band. It is a separate state meaning "the model could not make a prediction", it is never mapped to LOW, and the backend **rejects** it if a client tries to upload it.

---

11. Where inference runs

On the device, in the Android app. No network round-trip is needed to produce a prediction, so the tourist-facing feature works fully offline.

Decision	Reason
On-device, not a backend endpoint	A prediction that needs the network fails exactly when a tourist is out of signal. It also means no accelerometer stream ever leaves the phone.

Decision	Reason
Reuse `DecisionForest`	The app already walks the movement model's JSON forest. This model uses the **same** walker with `expectedFeatures = 22`. **No new ML framework, runtime or serialization format was added** - the only new dependency count is zero.
`expectedFeatures` parameter added to `DecisionForest.fromJson`	So a 22-feature model can load without weakening the existing 24-feature check for the movement model.
Same sensor subscription	The risk model is fed by the **existing** movement watch window (128 samples / 2.56 s, 0.64 s hop); it uses the **last 50 samples** of that window. No second sensor listener, no extra battery cost.
Inference separate from UI	`FutureRiskFeatures` -> `FutureRiskModel` -> `FutureRiskAnalyzer` produce a `FutureRiskState`; the Compose card renders a pure `FutureRiskCardContent` built by `FutureRiskCardModel`. Both layers are independently unit-tested; the wording layer needs no emulator.

Artefact size as shipped: **1,974,297 bytes** (120 trees, 66,894 nodes), i.e. the selected forest's 1,968,314 bytes plus ~6 KB of metadata (feature stats, definitions, calibration, thresholds, model card pointer).

---

12. Three different things, kept visibly separate

This is the requirement the feature was built around. The app shows all three, and each says what it is:

Shown as	What it actually is	Trained?	Has a probability?
Geofence status	A **configured rule**: inside/outside a polygon the authority drew	No	No
Approach risk (`ApproachRisk.kt`)	A **published weighted checklist** over crossing history	No - documented heuristic	No
Future risk prediction (this card)	A **trained random forest** over the last second of accelerometer data	**Yes**	Yes, calibrated

Consequences enforced in code and tests:

- Distinct types throughout (`FutureRiskBand` / `FutureRiskFactor` / `FutureRiskState` vs `RiskBand` / `RiskFactor`), so a screen cannot render the heuristic under the trained model's

heading.
- The card's own text names the other two and denies being them, and a test asserts that sentence exists.
- The card's footer states that the model **never sends an SOS by itself**, and a test asserts that too.
- **Risk never triggers SOS.** A HIGH band raises no SOS, marks nobody as "in trouble", and changes no operational state. `backend/tsm/api.py`'s `state` computation deliberately ignores risk entirely, and `backend/tests/test_risk.py` asserts it: a HIGH prediction leaves `latestSos` empty, `openSosCount == 0` and `state != "SOS"`.

---

13. What is shared with the authority, and when

A throttled **summary** - never a sensor stream - goes through the app's existing offline outbox as a new `OutboxKind.RISK` event (`origin = ON_DEVICE_MODEL`, `isSimulated = false`).

Policy (`FutureRiskReportPolicy`), all times measured on-device:

Trigger	Reason
First MODERATE/HIGH	`FIRST_ELEVATED`
Any escalation (MODERATE -> HIGH)	`ESCALATED` - never throttled
Unchanged band after 120 s	`REFRESHED`
Falling back to LOW, once	`CLEARED` - so the dashboard stops showing a warning that stopped being true
UNAVAILABLE or a stale band	never sent
LOW with nothing to clear	never sent

Payload (`OutboxPayloads.futureRisk`, validated field-by-field by `backend/tsm/payloads.py`):

`type` `RISK` | `timestampMillis` | `modelVersion` | `horizonSeconds` | `riskBand` (`LOW` | `MODERATE` | `HIGH` only) | `probability` (nullable) | `windowSamples` | `observedSamples` | `dataQuality{ok,samples,reason}` | `factors[<=5]{label,detail}`

The server rejects an unreadable band, a probability outside [0,1], a non-positive or absurd horizon (ceiling 600 s), a missing version, a malformed data-quality block, and more than 5 factors. Predictions are stored in their own `risk_assessments` table with their own read-back endpoints (`GET /api/v1/authority/risk`, per-tourist history, `/me.latestRisk`), and the dashboard shows them in their own panel and tiles - never folded into the SOS queue or the incident feed.

Dedupe key is the prediction instant (`risk:<timestampMillis>`), and the backend's event dedupe makes re-sending the same instant a `DUPLICATE` rather than a second row.

---

14. Tests

Suite	What it covers
`ml/future_risk/tests/test_future_risk.py` (93 tests)	Feature arithmetic on hand-computed signals; label geometry rebuilt from the real archive; leakage (no window contains its own impact, negatives only from ADL); subject-disjoint splits; Platt monotonicity and clipping; band rule; artefact schema, size budget, byte-identity with the app asset; reference cases; provenance including the dropped trial; tuning/selection rule; model card contents
`FutureRiskFeaturesTest` (13)	The 22 features in Kotlin, index by index, including the "no net direction" guard and the short-window refusal
`FutureRiskReferenceTest` (6)	The **shipped** asset against scikit-learn's own probabilities for 16 held-out windows: feature parity, forest parity, calibration parity, and refusal of an artefact with a wrong feature schema
`FutureRiskAnalyzerTest` (18)	Inference: band mapping at each threshold, calibrated-not-raw probability, insufficient data, implausible and non-finite readings, staleness, paused state, factor ordering, non-causal wording, determinism
`FutureRiskReportPolicyTest` (10)	Every reporting decision, including clock-jump robustness and one-clear-only
`FutureRiskReporterTest` (12)	Real SQLite outbox: one durable RISK row, summary-not-stream payload, no row when unconfigured, offline queue and recovery, dedupe
`FutureRiskPayloadTest` (8)	Wire contract through a real JSON parser: null probability, non-finite handling, escaping, no coordinates or raw samples
`FutureRiskMovementIntegrationTest` (8)	One sensor subscription for both models; no model -> "prediction unavailable"; a HIGH forecast raises no fall alert; implausible window refused by both; stop keeps the last forecast

Suite	What it covers
`FutureRiskCardModelTest` (18)	What the card may say: no band when there is no prediction, no "low" for "we do not know", thresholds printed, non-causal caveat, never claims to summon help
`backend/tests/test_risk.py` (28)	Ingest, idempotency, read-back, wire-level `PREDICTION` labelling, 12 validation rejections, and "a prediction is not an incident"
`backend/verify_integration.py` (33 checks)	Over a **real HTTP subprocess server**, an outbox batch shaped as the phone builds it carries a `RISK` summary: accepted, labelled `PREDICTION`, read back from `/authority/risk` and the tourist detail with its probability and factors, counted as elevated risk while `sosTotal` stays 1, and **survives a server restart**

Not executed: any on-device rendering or sensor behaviour. There is no emulator, AVD or physical device available in this environment (`adb devices` is empty), so the `FutureRiskCard` on a real screen, real permission dialogs, real GPS and the device HTTP stack remain **unverified**. All Android tests above are JVM/Robolectric tests.

---

15. Limitations - the complete list

1. **False alarms.** 5.09 per minute of ordinary activity at the app's cadence, and 16.3 per minute at the window stride. Not a deployable alarm (section 9).
2. **Lab data.** UniMiB SHAR is 30 healthy adults performing scripted activities in a lab, phone carried or pocketed. Nothing here was recorded from a tourist, outdoors, in weather, or with the phone in a bag or a moving vehicle.
3. **Not the best model in the repository.** `hist_gradient_boosting` reaches PR-AUC 0.8150 vs the shipped 0.7646 (section 9).
4. **1 s horizon only.** Not a 30-minute forecast, and no claim about any longer horizon.
5. **The label is a fall impact, not a tourist incident.** Every reported metric is about falls.
6. **Mid-range probabilities are over-confident** (section 8, bins 0.2-0.4).
7. **Activity-dependent false alarms** (jumping, sitting down, lying down).
8. **One class of failure is unmeasurable here:** the model cannot distinguish "a phone being dropped" from "a person falling" - both look like a hard impact followed by stillness. There is no phone-drop activity in the dataset, so this remains an untested failure mode.
9. **Not validated end-to-end on a device**, and possibly not faithful on-device: the Kotlin `DecisionForest` walk is proven against scikit-learn's numbers by test, but the whole chain (sensor -> window -> features -> forest -> card) has never run on real hardware here.

10. **Licence ambiguity** in the dataset (section 2).
11. **The 6 excluded fall trials and the 1 corrupt recording mean the training set is not the entire dataset.** Both are counted and reported rather than quietly dropped.

---

16. Reproducing this

```
# from the repository root
python ml/future_risk/train.py                 # full pipeline -> artefacts + app assets
python ml/future_risk/tune.py                  # the 65-config validation search (~10 min)
python -m pytest ml/future_risk/tests -q       # 88 pipeline tests

# or the whole ML suite, including this stage
powershell -NoProfile -ExecutionPolicy Bypass -File ml\run_all.ps1 -FutureRisk
```

`train.py` writes `future_risk_model.json` to `ml/outputs/future_risk/` **and** to `app/src/main/assets/`, plus `future_risk_reference_cases.json` to `app/src/test/resources/`. A test asserts the two copies are byte-identical, so the shipped model cannot drift from the trained one.

Versioning: `config.MODEL_VERSION` = `future-risk-unimib-v1` is embedded in the artefact, quoted in `metrics.json`, sent in every `RISK` payload, and shown in the app. Any change to features, label geometry, hyper-parameters or calibration requires a new version string.

Expected runtime: ~4 minutes for `train.py` on a modern desktop, dominated by reading `full_data.mat` and fitting the 120-tree forest.